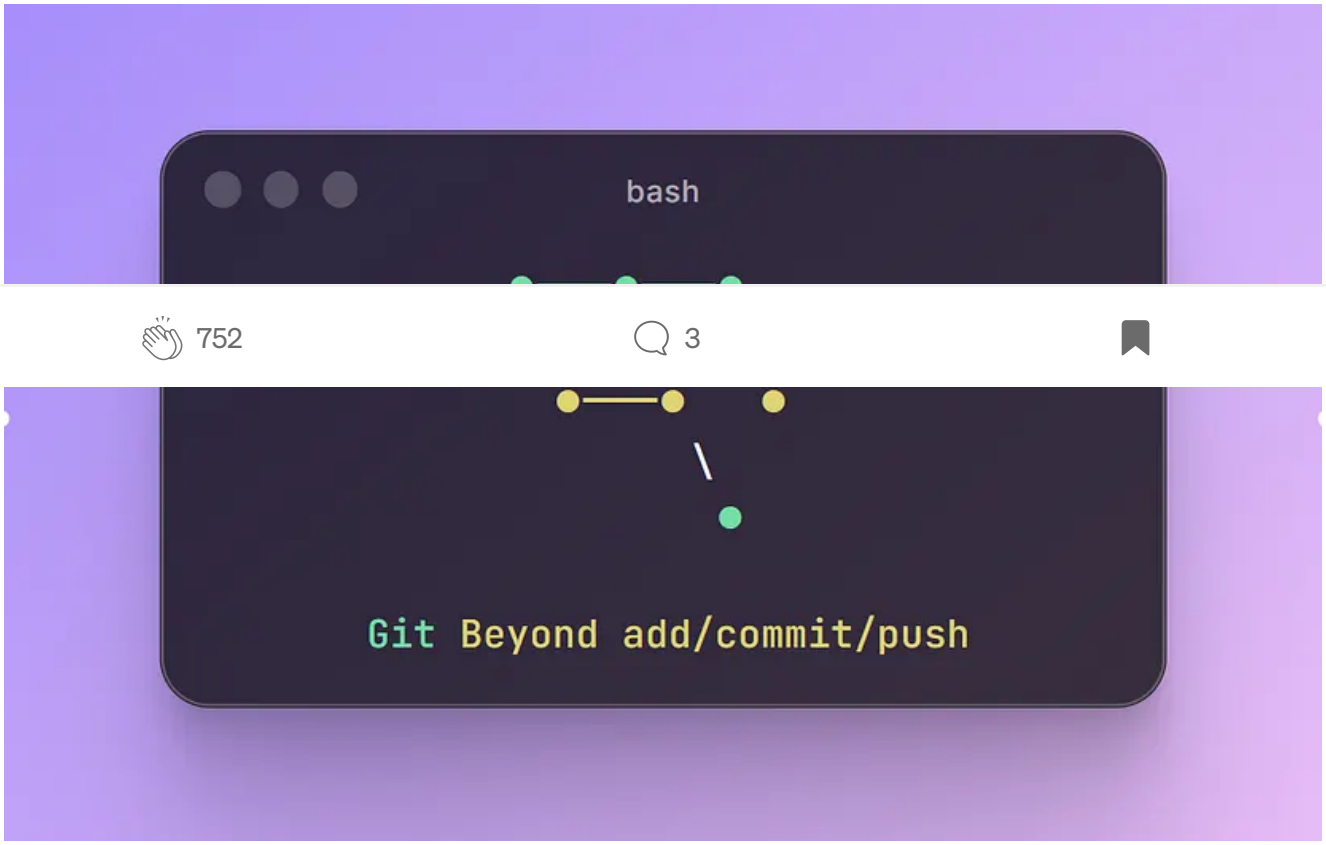
6 Commands That Quietly Changed How I Use Git

5 min read · Feb 15, 2026

Pushkar Singh Follow

Listen Share More



752 3

Every branch tells a story. The graph shows how they connect.

When I first started using Git, I treated it like a safety net.

```
# Modify files
git add .
git commit -m "update"
git push
```

And honestly, that was enough for a while.

But the moment I started working on larger repositories and trying to take Git more seriously, I began noticing small frustrations. Messy commits. Lost changes. Confusion while debugging. Too much branch switching.

That is when I realized I did not need more Git commands. I needed better control over the ones I already had access to.

These are six commands that slowly changed how I think about Git.

. . .

(1) The Day I Stopped Committing Everything Together

> `git add -p`

This is where I stopped committing files and started committing intentions.

There was a time when my commits looked like this:

“fixed bug + refactor + formatting”

It made sense when I wrote it. It made no sense three weeks later.

I remember once trying to trace a regression and realizing that my own commit history was the reason it was hard to understand what changed.

That was when I started using:

```
git add -p
```

The first few times felt slow. Git would show me small hunks of changes and ask whether I wanted to stage them.

Yes. No. Split. Edit.

It forced me to think.

Is this logically separate?
Should this be its own commit?

Over time, I stopped committing files. I started committing intentions.

And that changed how I approached writing code itself.

. . .

(2) When I Discovered I Could Search Through Time

> `git bisect`

Debugging stopped being guesswork once I started searching through history.

I still remember the first time something “mysteriously” broke.

It had worked before. I was sure of it.

I manually jumped across commits, checked out random points in history, tested things, got frustrated.

Then I learned about `git bisect`.

```
git bisect start
```

```
git bisect bad
git bisect good <commit>
```

Git checked out a midpoint commit. I tested it. Marked it good. It moved again.

It felt almost unfair.

Instead of randomly scanning history, Git was doing a binary search for me.

That moment shifted something in my head. History was no longer a linear scroll. It was a searchable space.

Now, whenever something regresses, I do not panic. I bisect.

And every time, I am reminded how powerful this tool really is.

. . .

(3) The First Time I Thought I Lost Everything

> `git reflog`

Most “lost” commits are not lost. They are just hiding behind moved pointers.

At some point, I ran:

```
git reset --hard
```

And immediately realized I should not have.

That sinking feeling is very real.

I thought the work was gone.

Then someone mentioned `git reflog`.

```
git reflog
```

I saw previous HEAD states listed there. Commits that were no longer visible in normal log output.

I reset back to one of them.

Everything came back.

That experience did more than save my work. It changed my confidence level with Git.

I stopped being scared of destructive commands. I started understanding that Git mostly moves references. It does not instantly destroy history.

Once that fear disappeared, experimenting became easier.

. . .

(4) The Stash-Switch Cycle That Was Wasting My Time

> `git worktree`

Parallel workflows without duplicating the repository.

There was a phase where my workflow looked like this:

Work on feature A.
Need to check something on another branch.
Stash. Switch. Test. Switch back. Pop stash.

Repeat.

It worked, but it felt clumsy.

Then I tried:

```
git worktree add ../feature-x feature-x
```

Suddenly I had another folder, tied to the same repository, but checked out to a different branch.

No stashing. No juggling.

Now I could keep one directory open for my main work and another for reviewing something else.

It felt like unlocking parallel development inside a single repo.

Once I started using worktrees, I could not go back.

. . .

(5) The Moment I Realized Stash Was Lying to Me

```
git stash push -u
```

Stash became reliable once I understood what it actually saves.

I used to think `git stash` saved everything.

It does not.

I once stashed changes, switched branches, and realized my untracked files were gone from my working directory and not in the stash either.

That confusion pushed me to look deeper.

```
git stash push -u
```

The `-u` includes untracked files.

I also discovered `--keep-index`, which lets me stash only what is not staged.

That is when stash stopped feeling unpredictable.

It became something I used intentionally, not casually.

. . .

(6) When I Needed to Know Why That Line Exists

> *`git blame -L`*

Sometimes you only need the history of a few lines.

In a larger codebase, I was staring at a block of logic and wondering:

Who wrote this?
And why?

Running plain `git blame` flooded my terminal with too much information.

Then I tried narrowing it:

```
git blame -L 120,150 file.c
```

Just the lines I cared about.

That precision made a huge difference.

Instead of scanning an entire file's history, I could focus on the exact region that mattered.

It made exploring unfamiliar code feel less overwhelming.

. . .

What Changed for Me

Individually, these commands solve specific problems.

But what really changed was my mindset.

I stopped treating Git as a backup tool.
I started treating it as a time control system.

I learned that:

- Commits should be intentional.
- Bugs can be searched algorithmically.
- Mistakes are often recoverable.
- Workflows can be structured instead of reactive.

The more I understood these commands, the calmer my development process became.

Less chaos.
Less fear.
More control.

And that, more than anything else, is what made me better at Git.

. . .

If you enjoyed reading this, read my other articles on ***Medium*** and follow me on ***Twitter(X)*** to continue the conversation.

Git | Software Development | Programming | Open Source | Debugging

Following

Published in Level Up Coding

307K followers · Last published 2 days ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Follow

Written by Pushkar Singh

83 followers · 8 following

Writing to learn, sharing to grow.

Responses (3)

To respond to this story,
get the free Medium app.

Continue in app

Jeevankumarkpl
Feb 23 (edited)

Wonderful article on rarely used extremely useful git commands. Keep writing such article.

15

Karthikeyan Sn
Feb 27

Good one

10

Nate Geslin
Feb 27

I think `git bisect` is one of the most useful and not well known git commands. I tell people about it all the time and almost always have never heard of it. Coupled with AI, git bisect can be an incredibly powerful tool to find when bugs were introduced and squash them. Even without AI, it's still super powerful.

9

More from the list: "GIT Gold"

Curated by Pancake Agree

In Stackad... by Subodh S...

15 Git Terms That Confuse Developers (an...

Oct 21, 2025

Bhavyansh

10 Git Commands Every Developer Should Know...

Oct 27, 2025

View list

More from Pushkar Singh and Level Up Coding

Pushkar Singh

Vim vs Nano: Should We Still Care in the Age of VS Code?

A quick comparison of two legendary terminal-based editors and why they still matter today.

May 14, 2025

In Level Up Coding by Christian Bernecker

Building an AI Agent from Scratch with pure Python

From Theory to Implementation: Building a Robust, Self-Correcting AI Agent from Scratch with Python

Feb 16 915 8

In Level Up Coding by Sergey Gromov

Best Open-Source Semantic Layer Tools in 2026

Best Open-Source Semantic Layer Tools in 2026: The Rise of the Metrics Layer in the Modern Data Stack

Mar 9 235 11

In Bootcamp by Pushkar Singh

Why I’m Focusing on Design as a Future Developer

I’m in my 2nd year of Computer Science, which means most of my time goes into coding, solving problems, and building projects. But over the...

Aug 23, 2025 13

See all from Pushkar Singh

See all from Level Up Coding

Recommended from Medium

Phil | Rentier Digital

I Stopped Vibe Coding and Started “Prompt Contracts” — Claude Code Went From Gambling to Shipping

Last Tuesday at 2 AM, I deleted 2,400 lines of code that Claude Code had just generated for me.

Feb 10 3.3K 90

Reza Rezvani

10 Claude Code Commands That Cut My Dev Time 60%: A Practical Guide

Custom slash commands, subagents, and automation workflows that transformed my team’s productivity—with copy-paste templates you can use

Nov 20, 2025 1.91K 35

In ITNEXT by Jacob Ferus

Cursor Is Dying

Cursor is a great product. It was one of the first great applications of AI in coding, moving past copy-pasting code from chats. But the AI...

Feb 11 575 30

Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software Engineer

Tools I use that are (usually) cheaper than Netflix

Jan 18 4.1K 99

In Level Up Coding by Teja Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brain Was Terrifying.

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who actually win.

Dec 28, 2025 11.3K 409

In Vibe Coding by Alex Dunlop

Claude Code's Creator, 100 PRs a Week—His Setup Will Surprise You

Simple principles most developers overlook completely

Jan 15 1.1K 28

See more recommendations